Sample guide and files for FE-dNN implementation in Abaqus/Standard

The sample files are for selected examples in:

Yuan Z, Yeoh K M, Poh L H 2026. An accessible and efficient finite element implementation for multiscale surrogate modeling using differential neural network setupyyy. Finite Elements in Analysis and Design, 257: 104542. https://doi.org/10.1016/j.finel.2026.104542

If you find the sample files useful / have used the files for your work, please cite the abovementioned paper. For academic use only. Thank you.

This example is provided as a reference to demonstrate how to use our framework through a simple peeling problem. The following sections briefly describe the included files and outline the step-by-step procedure to run the example. Users may also train their own neural network models and replace the provided parameter files to adapt the framework to their own simulation cases.

The following files are listed in the folder:

1) **Peeling.inp:** Abaqus input file for a 3D peeling problem between a rubber-like layer and a steel substrate. The rubber-like material is modeled using the dNN surrogate implemented via UMAT, while the steel substrate is described by Abaqus's built-in linear elastic model. The cohesive behavior is modeled using Abaqus's built-in traction–separation law. Geometric nonlinearity is activated.

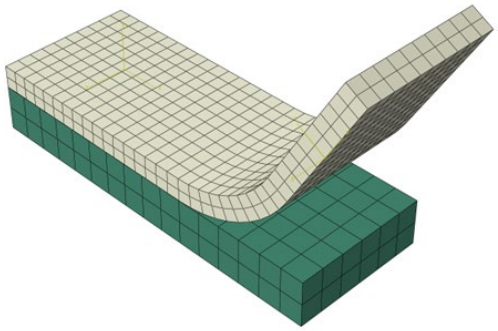

Fig 1. 3D peeling model

2) **Umat_v0.for:** V0 Fortran UMAT subroutine for the dNN surrogate model. The network parameters and normalization constants are loaded via EXTERNALDB and used in UMAT to reconstruct the dNN surrogate model for stress and tangent stiffness evaluation.
3) M**odel_nh.txt:** A text file containing the neural network parameters of the trained surrogate model. The provided sample corresponds to a homogeneous Neo-Hookean hyperelastic material, using material parameters E=10 and μ=0.3.
4) **Inputs/outputs_max/min.txt:** A text file storing the normalization constants for the input and output variables. In this example, the dataset is generated with principal stretches λ_i =[0.5,1.5].

How to run the example?

To run this example, users may follow the steps below:

Step 1: Place all provided files in the same working directory

Step 2: Open Umat_v0.for and modify the file paths

```
IF (LOP.EQ.0) THEN

  OPEN(15,file='D:\FE-dNN_example\inputs_min.txt')
  READ(15,*) INP_MIN
  CLOSE(15)

  OPEN(16,file='D:\FE-dNN_example\inputs_max.txt')
  READ(16,*) INP_MAX
  CLOSE(16)

  OPEN(17,file='D:\FE-dNN_example\outputs_min.txt')
  READ(17,*) OUT_MIN
  CLOSE(17)

  OPEN(18,file='D:\FE-dNN_example\outputs_max.txt')
  READ(18,*) OUT_MAX
  CLOSE(18)


  idx =(/0, 0/)
  OPEN(20,file='D:\FE-dNN_example\model_nh.txt')
```

Step 3：Submit the following command through the Abaqus command window:

abaqus job = Peeling.inp user = Umat_v0.for

Step 4：Check the results through the output database (.odb) file in Abaqus

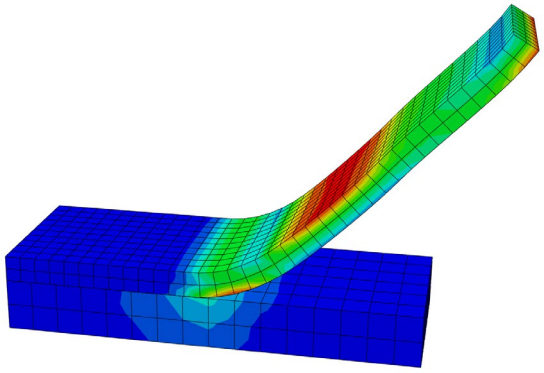

Fig 2. Simulation results of the peeling peoblem

How to adapt the framework to your own case?

To apply the framework to your own problem, the following modifications are required:

Step 1: Prepare your own training data and model

The NN surrogate can be trained using any machine learning framework of choice. In the current implementation, only the sigmoid activation function is supported. The model targets 3D large deformation hyperelasticity, taking the Green–Lagrange strain $\boldsymbol{E} \in \mathbb{R}^6$ as input and predicting the second Piola–Kirchhoff stress $\boldsymbol{S} \in \mathbb{R}^6$.

$$[E_{11}, E_{12}, E_{13}, E_{21}, E_{22}, E_{33}] \rightarrow [S_{11}, S_{12}, S_{13}, S_{21}, S_{22}, S_{33}]$$

Step 2. Normalize the data and export scaling constants

Before training, both strain and stress components need to be normalized using min–max scaling The corresponding minimum and maximum values for inputs and outputs are saved into text files: inputs_min.txt, inputs_max.txt, outputs_min.txt, and outputs_max.txt.

Step 3. Export model parameters

After training, export the neural network parameters (weights **W** and biases **b**) and save them in the file model_nh.txt. Next, update the network architecture in Umat_v0.for to match the trained model.

```
      PARAMETER (maxN = 256, maxL = 7)

      REAL(DP) INP_MIN(6), INP_MAX(6), OUT_MIN(6), OUT_MAX(6)
      REAL W(maxL+1, maxN, maxN+1)
      INTEGER Struc(maxL+2),idx(2),L
*********************************************************
      DATA L /4/, Struc /6, 128, 256, 256, 128, 6/
*************************************************************
```

Here, L denotes the number of hidden layers. The input and output dimensions are fixed to 6 for 3D problems. In the current implementation, the maximum number of hidden layers is 7, and each layer can contain up to 256 neurons. For the present example, the network consists of 4 hidden layers with the following sizes:

$$128 \rightarrow 256 \rightarrow 256 \rightarrow 128$$

In model_nh.txt, the network parameters are stored layer by layer from top to bottom. Each layer is represented by a weight matrix with the bias term embedded as an additional column. For the present example, the stored matrices follow the structure below:

	Input Size	Output Size	Matrix Size
Layer 1	6	128	128×7
Layer 2	128	256	256×129
Layer 3	256	256	256×257
Layer 4	256	128	128×257
Output Layer	128	6	6×129

Step 4. Prepare your Abaqus model

Generate the .inp file according to your own problem setup. Use 3D solid elements (C3D8) with full integration, select user material, and activate geometric nonlinearity.